just appended the head of a list to the tail of the other; sets, instead, forces us to actually add elements one by one. An extra linear number of assignments is needed (worst case), but this doesn't change the order of the function’s runtime: we still need to update the references for elements of one of the two lists (i.e. sets) anyway.

Here, we show the simplest code, always pouring the first partition's elements into the second one. On our repo on github you can find a slightly better version that checks which set is smaller and adds its elements to the larger sets: this is, however, just a constant-time improvement on the simplest version, and running time remains linear in the minimum of the sets sizes.

5.5 Using a Tree-like Structure[10]

To recap what we attained with our basic implementation: we managed to write a constant-time `findPartition` method and a worst-case linear-time `merge` method.

Now, can we do any better than linear, not just for `findPartition`, but for all the operations on a disjoint-set? Well, turns out that yes, we can!

5.5.1 From List to Trees

The idea is simple: instead of using lists to represent each partition, we will use trees, as shown in figure 5.7. Each